CHAPTER 13

Normalization, Denormalization & Data Modeling

Part III · Data & Storage — how to actually shape your tables.

LEARNING OBJECTIVES

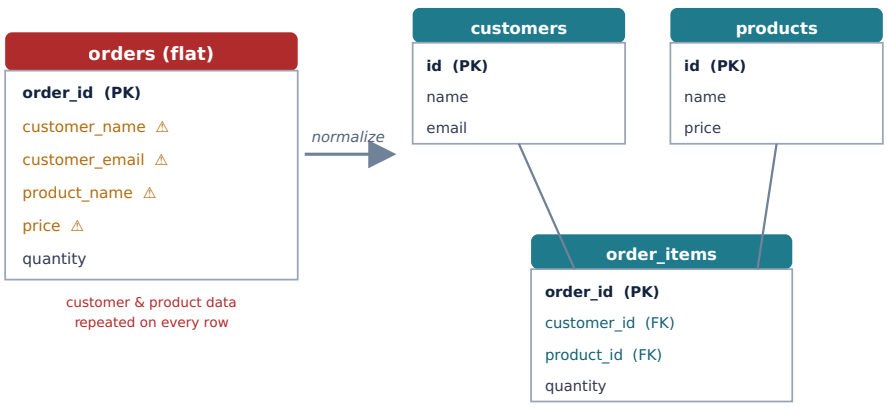
- Explain normalization and the data anomalies it prevents.
- Recognize the practical normal forms (1NF–3NF) without getting lost in theory.
- Explain denormalization and when adding redundancy is the right call.
- Weigh the normalize-vs-denormalize trade-off using the read/write ratio.
- Distinguish OLTP from OLAP and follow a practical data-modeling process.

REAL-WORLD ANALOGY — THE SHARED ADDRESS IN A CONTACT LIST

Suppose everyone at a company is in your contacts, and you write the company's full address on *every* person's card. When the company moves, you must edit dozens of cards — and if you miss one, your data now disagrees with itself. The tidy fix is to store the company's address **once** and have each person just reference the company. That is **normalization**. But sometimes, for speed, you deliberately keep a duplicate — a sticky note with the address right on a card so you don't have to look it up. That is **denormalization**. This chapter is about choosing between them.

13.1 The problem: how do you shape your tables?

Chapter 11 gave you tables and keys; Chapter 12 made queries fast. But *which* tables and columns should you create in the first place? Poor data modeling causes duplicated data, contradictions, and update headaches — or, at the other extreme, slow queries with too many JOINs. Good modeling balances two opposing forces: **normalization** (remove duplication for integrity) and **denormalization** (add duplication for speed).



13.2 What normalization is

Normalization is organizing data so each fact is stored **exactly once**, by splitting it into related tables. The goal is to prevent three **anomalies** that plague duplicated data:

- **Update anomaly:** a fact is duplicated across rows; you update one and the others go stale — the data now contradicts itself.
- **Insert anomaly:** you can't record one fact without also supplying unrelated data.
- **Delete anomaly:** deleting a row accidentally erases an unrelated fact stored only there.

13.3 The normal forms, practically

Form	Rule (in plain English)
1NF	Each cell holds a single, atomic value — no lists or repeating groups crammed into one column.
2NF	1NF, and every non-key column depends on the *whole* primary key (matters for composite keys).
3NF	2NF, and non-key columns depend only on the key — not on *other* non-key columns.

KEY INSIGHT — THE ONE-LINE SUMMARY

You can skip the formal theory and remember 3NF as: **"every non-key column depends on the key, the whole key, and nothing but the key."** Most real schemas aim for **3NF** — it removes the harmful redundancy without over-splitting. Higher forms exist (BCNF, 4NF) but are rarely the deciding factor in practice.

13.4 What denormalization is

Denormalization is the deliberate opposite: adding *controlled* redundancy back to speed up reads. Normalized data is clean, but answering a question often needs **JOINs** across several tables — and JOINs cost time (and become very painful once data is spread across shards, Chapter 17). Denormalizing avoids the JOIN by duplicating data or precomputing results:

- Store `author_name` directly on each `post` so showing a post needs no JOIN to `users`.
- Keep a precomputed `comment_count` on each post instead of running `COUNT(*)` on every read.
- Maintain a **materialized view** — a stored, pre-joined result that's refreshed periodically.

THE COST OF DENORMALIZING

Redundancy buys read speed but you now own the duplicates: every write must update **all copies**, which makes writes more complex and risks inconsistency if you miss one. You've traded the update anomaly back in, on purpose, in exchange for faster reads. Only do it where the read speedup genuinely matters and you can keep the copies in sync.

13.5 Choosing: normalize vs denormalize

Normalize when…	Denormalize when…
Data integrity and correctness are paramount.	A specific read path is hot and JOINs are the bottleneck.
The workload is write-heavy (fewer copies to keep in sync).	The workload is read-heavy and can tolerate slight staleness.
You're unsure — it's the safe default.	You've measured the JOIN cost and it's worth the write complexity.

KEY INSIGHT — NORMALIZE FIRST, DENORMALIZE SELECTIVELY

Recall Chapter 5: the read/write ratio drives this decision. Start **normalized** (correct and simple), then denormalize the *specific* hot read paths where measurement shows JOINs are too slow — keeping those duplicates carefully in sync. Denormalizing everything up front is premature optimization that fills your system with hard-to-maintain redundancy.

13.6 OLTP vs OLAP

The right balance also depends on what the database is *for*:

- **OLTP** (Online Transaction Processing): everyday app traffic — many small reads and writes, correctness-critical. Favors **normalized** schemas.
- **OLAP** (Online Analytical Processing): analytics and reporting over huge datasets — big read-only queries. Favors heavily **denormalized** designs (e.g. star schemas in a data warehouse) so reports don't JOIN dozens of tables.

Many companies run both: an OLTP database for the live app and a separate OLAP warehouse for analytics, fed from it.

13.7 A practical modeling process

1. **Understand the domain:** identify the entities and how they relate.
2. **Define tables and keys** for each entity and relationship.
3. **Normalize** to roughly 3NF — each fact once.
4. **Model for your access patterns:** shape the schema around the queries you'll actually run, not just tidy structure.
5. **Denormalize selectively** where measured read performance demands it.

Common mistakes

WATCH OUT

- Duplicating facts across rows and creating update anomalies (contradictory data).
- Denormalizing everywhere up front — premature optimization that's hard to maintain.
- Denormalizing but forgetting to keep the duplicated copies in sync on writes.
- Cramming lists/CSV values into one column (violating 1NF).

- Modeling for tidy structure while ignoring how the data will actually be queried.
- Using a normalized OLTP schema for heavy analytics instead of a separate OLAP store.

Interview questions

1. What is normalization, and what anomalies does it prevent?
2. Summarize 3NF in one sentence.
3. What is denormalization, and what does it cost?
4. How do you decide whether to normalize or denormalize a given part of a schema?
5. What's the difference between OLTP and OLAP, and how does each affect modeling?
6. Give an example of a denormalization that speeds up a read path and how you'd keep it consistent.

Hands-on exercises

1. Take a flat "orders" spreadsheet with repeated customer/product data and normalize it into tables.
2. Identify one update, one insert, and one delete anomaly in that flat table.
3. Propose one denormalization for a post-and-comments schema and describe how you'd keep it in sync.
4. Classify these as OLTP or OLAP: placing an order; a monthly revenue-by-region report.
5. For a "user profile page" read path, decide whether a denormalized `follower_count` is worth it.

Mini project

NORMALIZE, THEN DENORMALIZE WITH INTENT

Start from a messy, single-table export (orders with customer and product columns repeated on every row). Normalize it to 3NF as separate tables, and confirm the anomalies disappear. Then pick **one** hot read (say, "show an order with its customer name and product names"), measure the JOIN cost, and introduce a single, justified denormalization or materialized view to speed it up — documenting exactly how you'll keep the duplicate in sync on writes. You'll have practiced both directions and the judgment between them.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — REFINE THE MODEL

Review your Chapter 11 schema. Confirm it's normalized to ~3NF (no facts duplicated across rows). Then, using your read-heavy Chapter 5 numbers, choose **one or two** deliberate denormalizations for hot read paths — for example a stored `like_count` and `comment_count` on posts, and perhaps the author's display name cached on the post —

and write down the rule for keeping each in sync on writes. This is the schema you'll protect with transactions (Chapter 14) and later replicate and shard.

Chapter summary

- **Normalization** stores each fact once by splitting data into related tables, preventing update, insert, and delete **anomalies**.
- Most schemas target **3NF**: every non-key column depends on the key, the whole key, and nothing but the key.
- **Denormalization** deliberately adds redundancy to speed up reads — at the cost of harder writes and the risk of inconsistency.
- Choose by the **read/write ratio**: normalize by default; denormalize the specific hot read paths that measurement shows need it.
- **OLTP** (live app) favors normalized; **OLAP** (analytics) favors denormalized — often in a separate warehouse.
- Model for your **access patterns**, not just tidy structure.

COMING NEXT

Chapter 14 — Transactions, ACID & Isolation Levels. With a schema in place, we make changes to it *safely*. What happens when many users hit the same data at once? We'll go deep on ACID, the concurrency anomalies transactions prevent, and the isolation levels that dial correctness against performance.